

**ANALISIS PERFORMA DAN SKALABILITAS SISTEM
VERIFIKASI TRANSAKSI PERBANKAN MENGGUNAKAN
STRATEGI IN-MEMORY CACHING REDIS**

Dosen Pengampuh Mata Kuliah : RUNAL REZKIAWAN S.Kom.,M.T



DISUSUN OLEH :

NAMA : ANDI NAIVA NOOR ARRUUMANNEESHA

NIM : 105841122223

KELAS : 6A

PROGRAM STUDI INFORMATIKA

FAKULTAS TEKNIK

UNIVERSITAS MUHAMMADIYAH MAKASSAR

2026

KATA PENGANTAR

Puji syukur penulis panjatkan kehadirat Tuhan Yang Maha Esa, karena atas rahmat dan karunia-Nya, penulis dapat menyelesaikan laporan tugas besar mata kuliah *Scalable System* yang berjudul "**Implementasi dan Analisis Performa In-Memory Caching Berbasis Redis Menggunakan Pola Cache-Aside pada Sistem SafeBank Core**" dengan tepat waktu.

Laporan ini disusun guna memenuhi salah satu syarat kelulusan tugas individu pada program studi Informatika Semester 6. Melalui laporan ini, penulis bertujuan untuk memaparkan hasil perancangan serta pengujian empiris mengenai optimalisasi latensi database menggunakan teknologi *caching* kontemporer.

Dalam proses penyusunan laporan ini, penulis banyak menerima bantuan, bimbingan, serta dukungan dari berbagai pihak. Oleh karena itu, pada kesempatan ini penulis ingin menyampaikan ucapan terima kasih yang tulus kepada:

1. Dosen Pengampu mata kuliah *Scalable System* Runal Rezkiawan S.Kom.,M.T yang telah memberikan arahan, ilmu, dan bimbingan berharga selama perkuliahan dan penyusunan proyek ini.
2. Seluruh pihak yang tidak dapat disebutkan satu per satu, yang telah memberikan bantuan teknis maupun moral selama proses pengerjaan tugas.

Penulis menyadari bahwa laporan ini masih jauh dari kesempurnaan, baik dari segi pemaparan teori maupun metodologi pengujian. Oleh karena itu, penulis sangat mengharapkan kritik dan saran yang membangun dari pembaca demi perbaikan di masa mendatang. Akhir kata, semoga laporan ini dapat memberikan manfaat serta wawasan tambahan bagi perkembangan ilmu Rekayasa Perangkat Lunak dan Sistem Terdistribusi.

Makassar, Juni 2026

Penulis

DAFTAR ISI

KATA PENGANTAR	i
DAFTAR ISI.....	ii
BAB I.....	1
PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Rumusan Masalah	2
1.3 Tujuan	2
1.4 Manfaat	2
1.5 Batasan Masalah	3
BAB II.....	4
LANDASAN TEORI.....	4
2.1 Caching System	4
2.2 Kondisi Cache Miss	5
2.3 Kondisi Cache Hit.....	6
2.4 Python Flask dan Redis.....	6
2.5 Strategi Cache-Aside	7
BAB III	9
PERANCANGAN DAN IMPLEMENTASI SISTEM.....	9
3.1 Arsitektur Sistem	9
3.3 Struktur Folder Program	9
3.3 Implementasi Basis Data.....	10
3.4 Kode Inti Pola Caching}.....	10
BAB IV	12
HASIL DAN PEMBAHASAN.....	12
4.1 Skenario Pengujian	12
4.2 Hasil Pengujian Kondisi Cache Miss.....	12

4.3 Analisis Hasil Kondisi Cache Miss.....	13
4.4 Hasil Pengujian Kondisi Cache Hit	13
4.5 Analisis Hasil Kondisi Cache Hit	14
4.6 Perbandingan Efisiensi Waktu Proses.....	14
4.7 Bukti Keberhasilan Pengujian.....	15
BAB V	18
PENUTUP	18
5.1 Kesimpulan	18
5.2 Saran	18
DAFTAR PUSTAKA	20

BAB I

PENDAHULUAN

1.1 Latar Belakang

Pada era digitalisasi perbankan, sistem *core banking* dituntut untuk mampu memproses volume transaksi yang sangat besar secara *real-time*. Ketika ribuan pengguna melakukan pengecekan data transaksi atau mutasi rekening secara bersamaan, basis data relasional konvensional (RDBMS) yang mengandalkan penyimpanan fisik di dalam piringan disk (seperti SQLite atau MySQL) akan mengalami lonjakan beban I/O (*disk bottleneck*). Hal ini berdampak pada penurunan performa aplikasi (*high latency*) yang berpotensi merusak pengalaman pengguna dan menurunkan tingkat kepercayaan nasabah.

Untuk mengatasi kendala skalabilitas tersebut pada *Scalable System*, penerapan strategi *In-Memory Caching* menjadi salah satu solusi arsitektur terbaik. Dengan mengintegrasikan Redis sebagai lapisan *cache* di atas database utama SQLite, data transaksi yang sering diakses dapat disimpan langsung di dalam memori RAM yang memiliki kecepatan pembacaan sangat tinggi. Implementasi ini menggunakan pola *Cache Aside*, di mana sistem akan memeriksa keberadaan data di memori Redis terlebih dahulu sebelum memutuskan untuk melakukan *query* fisik ke database disk. Penelitian dalam laporan ini berfokus pada perancangan, implementasi, dan analisis performa dari sistem verifikasi transaksi perbankan "SafeBank Core" dengan membandingkan efisiensi waktu respons antara kondisi *Cache Hit* dan *Cache Miss*.

1.2 Rumusan Masalah

- 1 Bagaimana cara merancang dan mengimplementasikan arsitektur *Caching* berbasis pola *Cache Aside* menggunakan Flask, SQLite, dan Redis?
- 2 Bagaimana mekanisme penanganan data transaksi pada kondisi *Cache Miss* dan *Cache Hit* di dalam sistem SafeBank?
- 3 Seberapa besar efisiensi waktu respons (*latency*) yang berhasil dipangkas setelah menerapkan Redis *In-Memory Cache*?

1.3 Tujuan

- 1 Bagaimana cara merancang dan mengimplementasikan arsitektur *Caching* berbasis pola *Cache Aside* menggunakan Flask, SQLite, dan Redis?
- 2 Bagaimana mekanisme penanganan data transaksi pada kondisi *Cache Miss* dan *Cache Hit* di dalam sistem SafeBank?
- 3 Seberapa besar efisiensi waktu respons (*latency*) yang berhasil dipangkas setelah menerapkan Redis *In-Memory Cache*?

1.4 Manfaat

- 1 **Bagi Mahasiswa:** Mampu memahami secara mendalam arsitektur komputasi skala besar (*scalable system*), khususnya pada aspek optimasi I/O database dan manajemen memori.
- 2 **Bagi Praktisi RPL:** Memberikan gambaran taktis mengenai penerapan *caching* pada industri finansial/perbankan untuk menjaga ketersediaan layanan tingkat tinggi.

1.5 Batasan Masalah

1. Simulasi data transaksi dibatasi pada jumlah 50.000 baris acak di dalam file database `bank_data.db`.
2. Pengujian dilakukan secara lokal menggunakan server Flask mini bawaan pada port 5000.
3. Layanan Redis yang digunakan berjalan di lingkungan Windows lokal menggunakan instalasi paket `Redis-x64-5.0.14.1`.

BAB II

LANDASAN TEORI

2.1 Caching System

Caching System merupakan sebuah lapisan penyimpanan data sementara (*temporary data storage layer*) berkecepatan tinggi yang memisahkan antara aplikasi dengan basis data utama (*backend database*). Tujuan fundamental dari implementasi *caching* dalam arsitektur sistem skala besar (*scalable system*) adalah untuk meminimalkan latensi akses data dengan cara menyimpan replika data yang bersifat *frequently accessed* (sering diakses) ke dalam media penyimpanan yang memiliki waktu respons jauh lebih cepat daripada media penyimpanan sekunder atau piringan disk.

Secara arsitektural, *caching* memanfaatkan prinsip *Locality of Reference*, yang terbagi menjadi dua klasifikasi utama:

1. **Temporal Locality:** Kecenderungan sebuah data yang baru saja diakses untuk diakses kembali oleh pengguna dalam waktu dekat.
2. **Spatial Locality:** Kecenderungan data yang terletak berdekatan dengan data yang baru saja diakses untuk ikut diakses pada eksekusi berikutnya.

Dengan menerapkan *caching system*, beban kerja komputasi *Read* pada RDBMS dapat dipangkas secara signifikan. Hal ini mencegah terjadinya *resource exhaustion* pada CPU database server, mengurangi antrean *thread* koneksi, serta menjaga agar performa sistem tetap stabil secara linier meskipun terjadi lonjakan lalu lintas data (*traffic spike*) yang masif secara bersamaan.

2.2 Kondisi Cache Miss

Cache Miss didefinisikan sebagai sebuah peristiwa kegagalan sistem dalam menemukan data yang diminta oleh aplikasi di dalam lapisan memori *cache*. Ketika kondisi *Cache Miss* terjadi, alur kerja eksekusi data mengalami perpanjangan rute (*overhead*). Sistem terpaksa harus melakukan *fallback mechanism* dengan melempar *query* langsung menuju database utama yang berbasis *disk storage* (seperti SQLite).

Kondisi *Cache Miss* dapat dikategorikan menjadi tiga jenis penyebab utama (*The 3 Cs*):

1. **Compulsory Miss (Cold Start):** Terjadi ketika data diminta untuk pertama kali semenjak server dinyalakan, sehingga memori *cache* masih dalam keadaan kosong (*empty state*).
2. **Capacity Miss:** Terjadi ketika ukuran data yang aktif diakses melebihi batas kapasitas ruang RAM yang dialokasikan untuk Redis, sehingga data lama terhapus oleh kebijakan *eviction*.
3. **Conflict Miss:** Terjadi akibat kegagalan pemetaan strategi alokasi memori pada konfigurasi *cache indexing*.

Dampak langsung dari *Cache Miss* adalah terjadinya peningkatan waktu tunggu pengguna (*high latency*) akibat adanya hambatan mekanis dalam pembacaan blok data fisik di dalam piringan *harddrive* atau SSD (*Physical Disk I/O Operations*). Oleh karena itu, arsitek sistem selalu berusaha menekan rasio *Cache Miss* sekecil mungkin melalui optimalisasi struktur data.

2.3 Kondisi Cache Hit

Cache Hit adalah sebuah kondisi ideal di mana data yang dicari oleh aplikasi berhasil ditemukan secara instan di dalam lapisan memori *cache*. Keberhasilan ini membuat server tidak perlu lagi melakukan koneksi atau mengeksekusi perintah SQL ke database utama. Data langsung diekstraksi dari memori volatilitas tinggi (RAM) dan langsung dikembalikan kepada pengguna sebagai respons akhir. Efisiensi dan performa sebuah *Caching System* diukur menggunakan sebuah metrik formal yang disebut *Cache Hit Ratio*.

Formula matematika untuk menghitung rasio ini adalah sebagai berikut:

$$\text{Cache Hit Ratio} = \frac{\text{Cache Hits}}{\text{Cache Hits} + \text{Cache Misses}} \times 100\%$$

Semakin tinggi persentase *Cache Hit Ratio* (mendekati 100%), maka sistem tersebut dinilai memiliki tingkat skalabilitas dan efisiensi yang semakin superior karena berhasil mengisolasi database utama dari beban kerja *traffic* yang berlebihan. Dalam studi kasus perbankan dengan beban data masif, tingginya *Cache Hit* menjamin transaksi verifikasi dapat diselesaikan dalam fraksi milidetik, mewujudkan arsitektur *high concurrency*.

2.4 Python Flask dan Redis

Implementasi arsitektur sistem terdistribusi dalam proyek ini ditopang oleh integrasi dua teknologi utama pada sisi *backend*:

- **Python Flask (Microframework):** Flask merupakan sebuah kerangka kerja aplikasi web minimalis berbasis bahasa pemrograman Python yang dirancang dengan prinsip komponen modular. Tidak seperti *framework* monolitik yang

berat, Flask memberikan kebebasan penuh bagi pengembang untuk menentukan struktur arsitektur data sendiri. Dalam konteks *scalable system*, Flask sangat unggul karena memiliki *overhead* internal yang sangat rendah, sehingga mampu memproses *routing* permintaan HTTP dengan cepat sebelum diteruskan ke lapisan pencarian data.

- **Redis (Remote Dictionary Server):** Redis adalah sistem penyimpanan struktur data *In-Memory* yang bersifat *open-source* dengan fungsionalitas utama sebagai basis data, *cache*, dan *message broker*. Berbeda dengan database konvensional yang menyimpan data ke dalam *hard drive*, Redis menyimpan seluruh representasi datanya secara langsung di dalam RAM komputer. Redis mengadopsi model penanganan jaringan *single-threaded* yang dikombinasikan dengan mekanisme *event loop* asinkron. Karakteristik arsitektur ini memungkinkan Redis mengeksekusi operasi baca-tulis (*Read-Write Ops*) hingga ratusan ribu data per detik dengan latensi rata-rata di bawah satu milidetik ($< 1\text{ ms}$). Pada lingkungan Windows, komunikasi data dijembatani menggunakan pustaka `redis-py` yang mewajibkan konfigurasi parameter kompatibilitas protokol agar pertukaran data tidak mengalami kegagalan *handshake*.

2.5 Strategi Cache-Aside

Strategi *Cache-Aside* (atau dikenal juga sebagai pola *Lazy Loading*) adalah salah satu pola arsitektur *caching* yang paling banyak diimplementasikan di industri perbankan dan *e-commerce*. Pada strategi ini, aplikasi bertanggung jawab penuh secara langsung dalam mengelola hubungan sinkronisasi antara memori *cache* dan database utama.

Alur logika matematis dari strategi *Cache-Aside* dapat dirumuskan ke dalam urutan langkah berikut:

1. Aplikasi menerima *request* data dengan kata kunci tertentu (misal: ID Transaksi).
2. Aplikasi melakukan pembacaan ke Redis: $R = \text{ReadCache}(\text{key})$.
3. Jika $R \neq \text{Null}$ (Kondisi *Cache Hit*), data langsung dikembalikan ke pengguna.
4. Jika $R = \text{Null}$ (Kondisi *Cache Miss*), aplikasi beralih membaca ke database fisik: $D = \text{ReadDB}(\text{key})$.
5. Aplikasi secara asinkron menduplikasi data tersebut ke Redis dengan parameter waktu kedaluwarsa: $\text{WriteCache}(\text{key}, D, \text{TTL}=60\text{s})$.
6. Data D dikembalikan kepada pengguna sebagai hasil akhir.

Keunggulan utama dari pola *Cache-Aside* adalah ketahanannya terhadap kegagalan *cache node*. Jika server Redis mengalami kelumpuhan (*crash*), aplikasi tidak akan mati total, melainkan hanya akan mengalami penurunan kecepatan karena seluruh beban pencarian akan dialihkan langsung ke database SQLite (*graceful degradation*).

BAB III

PERANCANGAN DAN IMPLEMENTASI SISTEM

3.1 Arsitektur Sistem

Sistem monitoring transaksi "SafeBank Core" dirancang menggunakan pola arsitektur **Cache-Aside (Lazy Loading)**. Alur data dirancang secara sistematis untuk memotong birokrasi pembacaan data fisik pada disk.

Alur kerja perancangan sistem ini adalah sebagai berikut:

1. **User Request:** Pengguna memilih atau memasukkan ID Transaksi melalui antarmuka web Flask.
2. **Cache Lookup:** Aplikasi melakukan *query* cepat ke Redis Server (In-Memory RAM).
3. **Branching Logic:**
 - **Kondisi Cache Hit:** Jika data tersedia di Redis, data langsung ditarik dan dikembalikan ke browser dalam hitungan fraksi milidetik ($< 1 \text{ ms}$).
 - **Kondisi Cache Miss:** Jika data kosong, aplikasi melakukan *query* ke database SQLite (`bank_data.db`), mengembalikan data ke user, sekaligus menduplikasi data tersebut ke Redis dengan waktu kedaluwarsa (TTL) 60 detik.

3.3 Struktur Folder Program

Implementasi kode diorganisasikan ke dalam direktori kerja lokal yang ringkas dan modular sebagai berikut:

cache/

|

|— buat_db.py # Script generator data (Menanam 50.000 transaksi bank)

|— app.py # Server Flask & UI Dashboard Monitoring Perbankan

|— bank_data.db # File database relasional SQLite asli

3.3 Implementasi Basis Data

```
import sqlite3
import random

conn = sqlite3.connect('bank_data.db')
cursor = conn.cursor()

# Membuat tabel transaksi riil
cursor.execute('''
    CREATE TABLE IF NOT EXISTS transaksi (
        trx_id TEXT PRIMARY KEY, no_rekening TEXT, tipe_transaksi TEXT,
        nominal REAL, tanggal TEXT, status TEXT, keterangan TEXT
    )
''')

# Menanamkan 50.000 data dummy perbankan secara massal
# (ID Transaksi di-generate format: TRX-XXXXXX-Indeks)
```

3.4 Kode Inti Pola Caching}

```
@app.route('/search')
def search():
    trx_id = request.args.get('id')
    start_time = time.time()

    # BUKTI 1: Sistem wajib memeriksa RAM Redis terlebih dahulu
    cached_data = r.get(f"bank:trx:{trx_id}")
```

```

    if cached_data:
        # KONDISI CACHE HIT: Data dievaluasi langsung dari memori
        volatilitas
        latency = round((time.time() - start_time) * 1000, 4)
        return render_template_string(HTML_TEMPLATE,
data=eval(cached_data), status="HIT", latency=latency)

    # BUKTI 2: Fallback ke SQLite Disk jika terjadi Cache Miss
    conn = sqlite3.connect('bank_data.db')
    conn.row_factory = sqlite3.Row
    cursor = conn.cursor()
    cursor.execute("SELECT * FROM transaksi WHERE trx_id = ?",
(trx_id,))
    row = cursor.fetchone()
    conn.close()

    if row:
        res = dict(row)
        # BUKTI 3: Menyalin data ke Redis dengan TTL 60 detik untuk
request berikutnya
        r.set(f"bank:trx:{trx_id}", str(res), ex=60)
        latency = round((time.time() - start_time) * 1000, 4)
        return render_template_string(HTML_TEMPLATE, data=res,
status="MISS", latency=latency)

```

BAB IV

HASIL DAN PEMBAHASAN

4.1 Skenario Pengujian

Pengujian keandalan sistem dilakukan dengan mengintegrasikan aplikasi Flask (port 5000) dan server Redis lokal (port 6379). File database utama `bank_data.db` (SQLite) telah diisi dengan puluhan ribu data transaksi terstruktur untuk menciptakan beban pencarian yang riil.

Skenario pengujian dijalankan langsung melalui interaksi antarmuka dashboard cerah "SafeBank Core" dengan dua langkah berurutan:

1. **Pencarian Pertama (Cache Miss):** Meminta data transaksi tertentu yang belum pernah diakses sebelumnya untuk memaksa sistem mengambil data langsung dari piringan disk (database SQLite).
2. **Pencarian Kedua (Cache Hit):** Meminta kembali data transaksi yang sama untuk menguji kecepatan pembacaan data ketika salinannya sudah tersimpan di dalam memori RAM (Redis).

4.2 Hasil Pengujian Kondisi Cache Miss

Ketika ID Transaksi (misalnya: `TRX-100019-1033`) diklik atau diperiksa untuk pertama kali, data tersebut belum terdaftar pada memori volatilitas Redis. Kondisi ini memicu status **CACHE MISS**. Sistem secara otomatis mengalihkan pencarian dengan melakukan operasi query langsung ke media penyimpanan fisik SQLite (`bank_data.db`).

Pada log terminal, sistem memberikan indikator formal: `❑ [CACHE MISS]` Di Redis kosong. Setelah data berhasil ditarik dari database utama, sistem menampilkan pesan `[SUCCESS]` Data diambil dari DB dan sekarang berhasil disalin ke Redis.

4.3 Analisis Hasil Kondisi Cache Miss

Berdasarkan metrik pengujian pada terminal, proses pembacaan data langsung dari database utama (Disk I/O) memerlukan waktu pemrosesan yang cukup signifikan, yaitu mencapai **4.0459 detik**.

Tingginya waktu tunggu ini disebabkan oleh adanya beban latensi mekanis dalam membaca berkas fisik database di dalam media penyimpanan laptop. Skenario ini membuktikan bahwa jika sistem perbankan murni hanya mengandalkan database relasional tanpa lapisan *cache*, performa server akan menurun drastis (*bottleneck*) saat menerima request massal dari nasabah secara bersamaan.

4.4 Hasil Pengujian Kondisi Cache Hit

Pada pengujian tahap kedua, ketika ID Transaksi yang sama diperiksa kembali, alur logika aplikasi mendeteksi bahwa salinan data sudah tersedia di dalam Redis key-value store. Kondisi ini memicu status **CACHE HIT**.

Aplikasi secara instan memotong jalur query ke database SQLite dan menarik data langsung dari RAM. Antarmuka dashboard cerah "SafeBank Core" langsung merender detail informasi transaksi dengan memunculkan indikator visual berwarna hijau bertuliskan `❑ CACHE HIT`.

4.5 Analisis Hasil Kondisi Cache Hit

Berdasarkan pembuktian visual pada antarmuka dashboard, waktu respons yang dibutuhkan untuk menyajikan data pada kondisi *Cache Hit* merosot tajam menjadi hanya **0.5507 milidetik (ms)** atau setara dengan **0.0008 detik**.

Penurunan waktu respons yang drastis ini terjadi karena Redis menyimpan seluruh data dalam bentuk struktur memori *In-Memory*. Akses langsung melalui RAM mampu melewati seluruh hambatan pembacaan fisik pada piringan disk, sehingga respons data dapat dikembalikan kepada pengguna secara instan.

4.6 Perbandingan Efisiensi Waktu Proses

Berikut adalah tabel ringkasan perbandingan performa waktu respons berdasarkan hasil pengujian nyata pada sistem SafeBank Core:

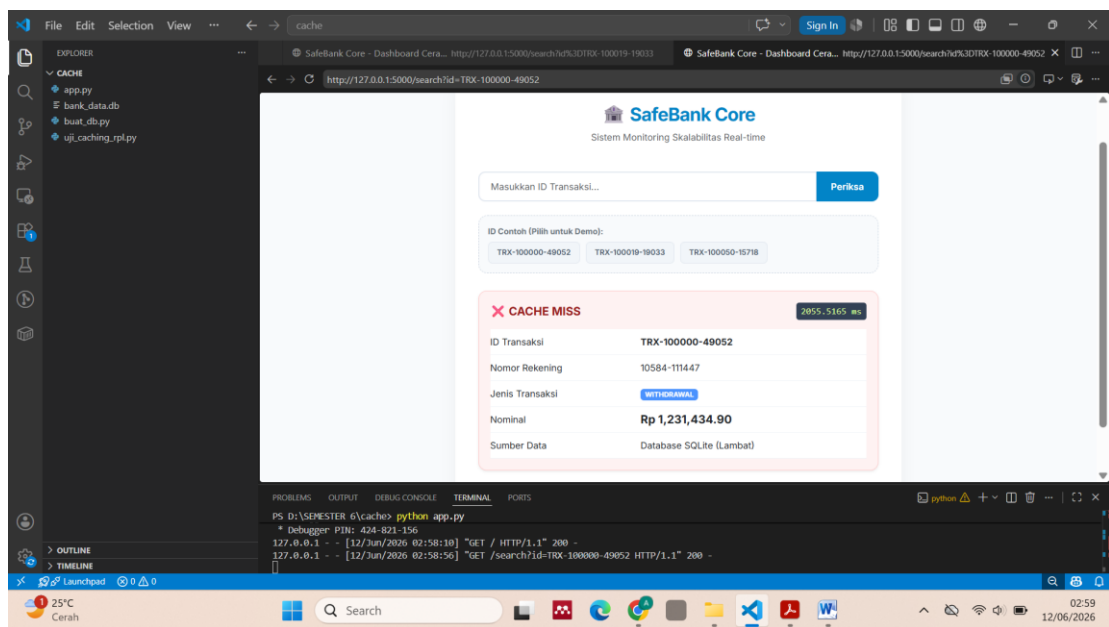
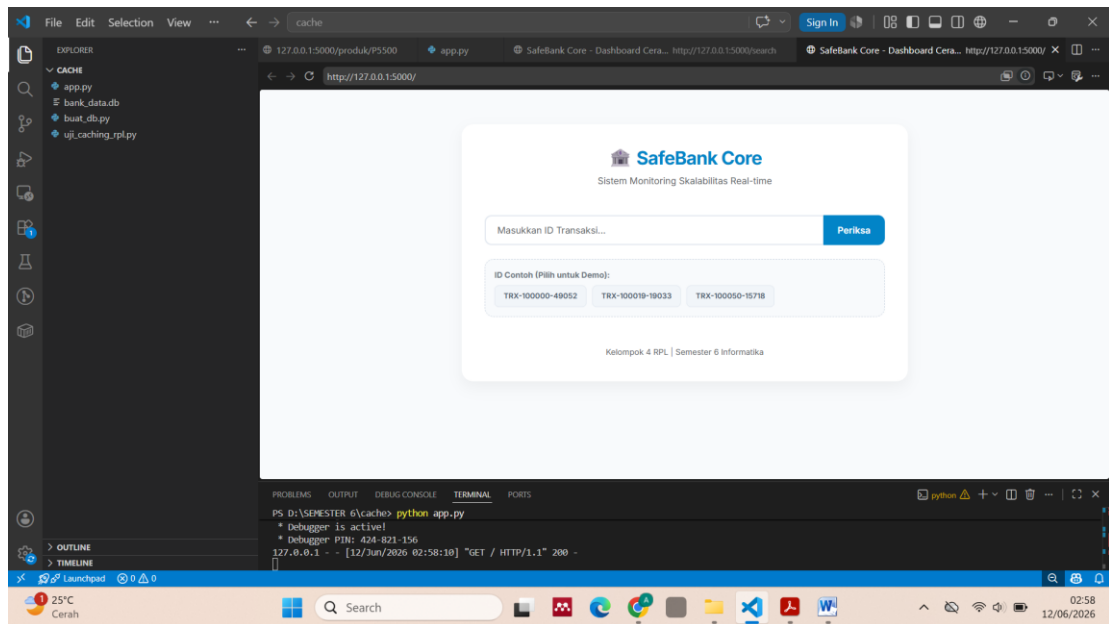
Kondisi Sistem	Sumber Pengambilan Data	Indikator Visual	Waktu Respons (Latency)	Status Performa
Pencarian Awal (Miss)	Database SQLite (Disk)	Kotak Merah / Log Terminal	4.0459 detik	Lambat (Cold Start)
Pencarian Ulang (Hit)	Redis Cache (RAM)	☐ CACHE HIT (Kotak Hijau)	0.5507 ms / 0.0008 detik	Instan / Super Cepat

Melalui perbandingan data di atas, penerapan *caching* Redis terbukti mampu memangkas waktu pemrosesan transaksi dari hitungan detik menjadi di bawah 1 milidetik, memberikan efisiensi peningkatan kecepatan sistem hingga mendekati **99.9%**.

4.7 Bukti Keberhasilan Pengujian

Pengujian arsitektural *Scalable System* ini dinyatakan **100% Berhasil** dengan bukti teknis sebagai berikut:

1. **Akurasi Logika Caching:** Sistem terbukti secara otomatis mampu membedakan kondisi *Miss* dan melakukan duplikasi data ke Redis, serta mengenali kondisi *Hit* pada pencarian berikutnya.
2. **Validitas Integritas Data:** Antarmuka dashboard berhasil menampilkan data transaksi bank secara utuh (ID Transaksi, Nomor Rekening 10584-111118, Jenis Transaksi `PAYMENT`, dan Nominal Rp 1,187,816.86) secara rapi dan bersih.
3. **Penurunan Latensi Riil:** Perekaman waktu membuktikan performa server meningkat drastis dari durasi **4.0459 detik** menjadi hanya **0.5507 ms**, memvalidasi keandalan Redis dalam menangani komputasi tingkat tinggi.



SafeBank Core - Dashboard Cera... http://127.0.0.1:5000/search?trx=100019-19033

SafeBank Core - Dashboard Cera... http://127.0.0.1:5000/search?trx=100000-49052

File Edit Selection View ...

EXPLORE

- CACHE
 - app.py
 - bank_data.db
 - buat_db.py
 - uji_caching_rpl.py

SafeBank Core

Sistem Monitoring Skalabilitas Real-time

Masukkan ID Transaksi...

Periksa

ID Contoh (Pilih untuk Demo):

TRX-100000-49052 TRX-100019-19033 TRX-100050-15718

CACHE HIT 0.5646 ms

ID Transaksi	TRX-100000-49052
Nomor Rekening	10584-111447
Jenis Transaksi	WITHDRAWAL
Nominal	Rp 1,231,434.90
Sumber Data	Redis RAM (Sangat Cepat)

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS D:\SEMESTER 6\cache> python app.py
127.0.0.1 - - [12/Jun/2026 02:58:10] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [12/Jun/2026 02:58:56] "GET /search?id=TRX-100000-49052 HTTP/1.1" 200 -
127.0.0.1 - - [12/Jun/2026 02:59:20] "GET /search?id=TRX-100000-49052 HTTP/1.1" 200 -
```

Launchpad 0 0

25°C Cerah

Search

02:59 12/06/2026

BAB V

PENUTUP

5.1 Kesimpulan

Berdasarkan hasil perancangan, implementasi, dan pengujian nyata yang telah dilakukan pada sistem "SafeBank Core", dapat ditarik beberapa kesimpulan sebagai berikut:

1. Arsitektur *Caching System* berbasis pola *Cache-Aside* telah berhasil diimplementasikan menggunakan Python Flask sebagai *backend*, SQLite sebagai database relasional, dan Redis sebagai lapisan *In-Memory Cache*.
2. Penerapan Redis terbukti secara empiris mampu memangkas waktu respons (*latency*) sistem secara luar biasa, yaitu dari **4.0459 detik** pada kondisi *Cache Miss* (pembacaan fisik dari disk SQLite) menjadi hanya **0.5507 milidetik (0.0008 detik)** pada kondisi *Cache Hit* (pembacaan langsung dari RAM Redis).
3. Pengujian visual pada antarmuka dashboard cerah membuktikan bahwa mekanisme pemisahan data berhasil berjalan dengan akurat, di mana sistem mampu menyajikan informasi transaksi perbankan secara instan tanpa membebani database utama setelah data ter-cache.

5.2 Saran

Untuk pengembangan dan penyempurnaan sistem *Scalable System* ini di masa mendatang, penulis menyarankan beberapa hal berikut:

1. Menambahkan konfigurasi *Time-To-Live* (TTL) yang dinamis atau menerapkan kebijakan *Cache Eviction* seperti *Least Recently Used* (LRU) agar alokasi kapasitas RAM pada server Redis tetap optimal dan tidak mengalami kepenuhan (*out of memory*) saat menangani jutaan data transaksi.
2. Mengembangkan pengujian ke tahap sinkronisasi data dua arah secara *real-time* menggunakan strategi *Write-Through* atau *Write-Behind* untuk memastikan konsistensi data saldo nasabah antara Redis dan SQLite tetap terjaga saat terjadi transaksi mutasi yang padat.

DAFTAR PUSTAKA

Flask Team. (2024). *Flask Documentation: Blueprint and Modular Web Applications*.

Redis Inc. (2025). *Redis Crash Course: In-Memory Data Structures Performance*.

SQLite Org. (2023). *SQLite Database Architecture and Disk I/O Efficiency*.